



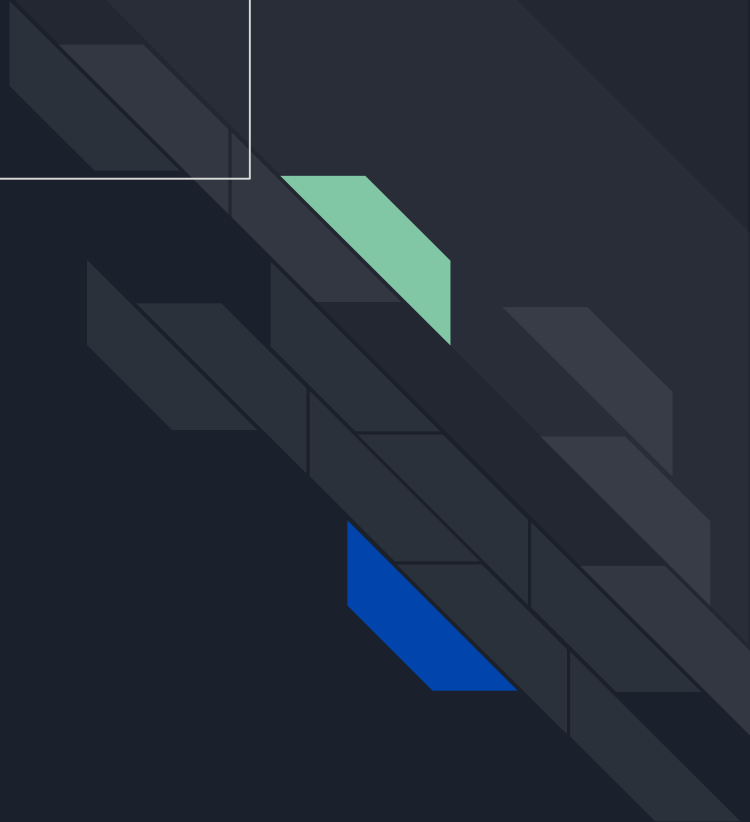
“Inspiring Minds”

Problem Solving and
Programming Fundamentals
CCC 1123





Pointer





Pointer

Pointers are powerful features of C and C++ programming.

Before we learn pointers, let's learn about addresses in C programming.

Address in C

If you have a variable `var` in your program, `&var` will give you its address in the memory. We have used address numerous times while using the `scanf()` function.

```
scanf("%d", &var);
```

Here, the value entered by the user is stored in the address of `var` variable. Let's take a working example.

```
#include <stdio.h>
int main() {
    int var = 5;
    printf("var: %d\n", var);
    // Notice the use of & before var
    printf("address of var: 0x%p", &var);
    return 0;
}
```

Output:

var: 5

address of var: hexadecimal number

Memory Address

- When a variable is created in C, a **memory address** is assigned to the variable.
- The memory address is the **location of where the variable is stored** on the computer.
- When we assign a value to the variable, it is stored in this memory address.
- **To access it, use the reference operator (&),** and the result represents where the variable is stored.
- **Note:** The memory address is in **hexadecimal** form (0x.). You will probably not get the same result in your program, as this depends on where the variable is stored on your computer.

```
#include <stdio.h>

int main() {
    int myAge = 43;
    printf("%p", &myAge);
    return 0;
}
```

```
// Outputs 0x7ffe5367e044
```



Pointer

- A pointer is a variable that stores the memory address of another variable as its value.
- A pointer variable points to a data type (like int) of the same type, and is created with the *** operator**.
- In the previous code, you should also note that **&myAge** is also called a "pointer".
- A pointer basically stores the memory address of a variable as its value.
- To print pointer values, we use the %p format specifier.

```
// Outputs 0x7ffe5367e044
```



Cont..

Pointers (pointer variables) are special variables that are used to store addresses rather than values.

Syntax:

```
int *p1, p2;
```

Here, we have declared a **pointer p1** and a **normal variable p2**.

Two ways to declare a pointer variable.

```
int* myNum;  
int *myNum;
```


Cont..

- A pointer variable points to a data type (like int) of the same type, and is created with the *** operator**.
- The address of the variable you are working with is assigned to the pointer:

```
int myAge = 43;      // An int variable
int* ptr = &myAge;  // A pointer variable, with the name ptr, that stores the address
                    // of myAge

// Output the value of myAge (43)
printf("%d\n", myAge);

// Output the memory address of myAge (0x7ffe5367e044)
printf("%p\n", &myAge);

// Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);
```

43
0x7ffcc5576d74
0x7ffcc5576d74

Cont..

- You can also get the **value of the variable the pointer points to**, by using the * operator

```
int myAge = 43;    // Variable declaration
int* ptr = &myAge; // Pointer declaration

// Reference: Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);

// Dereference: Output the value of myAge with the pointer (43)
printf("%d\n", *ptr);
```

0x7ffe5367e044

43

Pointer to Array

- In C, the name of an array, is actually a pointer to the first element of the array.
- The memory address of the first element is the same as the name of the array.
- It means, we can work with arrays through pointers

```
int myNumbers[4] = {25, 50, 75, 100};  
  
// Get the memory address of the myNumbers array  
printf("%p\n", myNumbers);  
  
// Get the memory address of the first array element  
printf("%p\n", &myNumbers[0]);
```

Result:

```
0x7ffe70f9d8f0  
0x7ffe70f9d8f0
```

Cont..

- Since **myNumbers** is a pointer to the first element in the array, you can use the ***** operator to access it.
- To access the rest of the elements in **myNumbers**, you can increment the pointer/array (+1, +2, etc).

```
int myNumbers[4] = {25, 50, 75, 100};
```

```
// Get the value of the first element in myNumbers  
printf("%d", *myNumbers);
```

25

```
int myNumbers[4] = {25, 50, 75, 100};
```

```
// Get the value of the second element in myNumbers  
printf("%d\n", *(myNumbers + 1));
```

```
// Get the value of the third element in myNumbers  
printf("%d", *(myNumbers + 2));
```

50

75

Using loop in array and pointer

```
int myNumbers[4] = {25, 50, 75, 100};  
int *ptr = myNumbers;  
int i;  
  
for (i = 0; i < 4; i++) {  
    printf("%d\n", *(ptr + i));  
}
```

25
50
75
100

Update element in array and pointer

```
int myNumbers[4] = {25, 50, 75, 100};
```

```
// Change the value of the first element to 13
```

```
*myNumbers = 13;
```

```
// Change the value of the second element to 17
```

```
*(myNumbers + 1) = 17;
```

```
// Get the value of the first element
```

```
printf("%d\n", *myNumbers);
```

13

```
// Get the value of the second element
```

```
printf("%d\n", *(myNumbers + 1));
```

17



Assigning addresses to Pointers

```
int *pc, c;
```

```
c = 5;
```

```
pc = &c;
```

Here, 5 is assigned to the c variable. And, the address of c is assigned to the pc pointer.

```
printf("%d", *pc); // Output: 5
```

Here, the address of c is assigned to the pc pointer. To get the value stored in that address, we used *pc.

Note: In the above example, pc is a pointer, not *pc. You cannot and should not do something like *pc = &c; By the way, * is called the dereference operator (when working with pointers). It operates on a pointer and gives the value stored in that pointer.

Changing Value Pointed by Pointers

```
int *pc, c;
```

```
c = 5;
```

```
pc = &c;
```

```
c = 1;
```

```
printf("%d", c); // Output: 1
```

```
printf("%d", *pc); // Output: 1
```

We have assigned the address of c to the pc pointer.

Then, we changed the value of c to 1. Since pc and the address of c is the same, *pc gives us 1.

Changing Value Pointed by Pointers

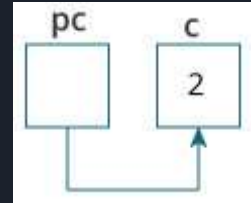
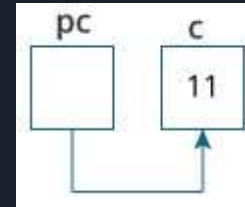
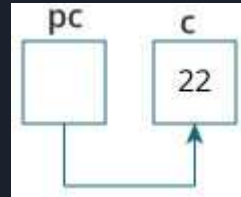
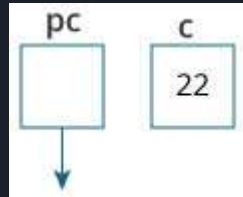
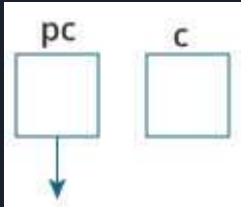
```
int *pc, c;  
c = 5;  
pc = &c;  
*pc = 1;  
printf("%d", c); // Output: 1  
printf("%d", *pc); // Output: 1
```

We have assigned the address of c to the pc pointer.

Then, we changed *pc to 1 using *pc = 1;. Since pc and the address of c is the same, c will be equal to 1.

Explanation of the program

1. `int *pc, c;`
2. `c = 22;`
3. `pc = &c;`
4. `c = 11;`
5. `*pc = 2;`





Goals for next session

1. Final Project

Thank You

The background features a series of dark gray, three-dimensional rectangular planes that recede into the distance, creating a sense of depth. A light green parallelogram is positioned on one of the upper planes, and a blue parallelogram is on a lower plane, both adding a pop of color to the monochromatic scheme.